

Jeu d'Échec Yalta — Rapport technique

Gaël Amoros

Avril 2026

1 Contexte et analyse du problème

Le Yalta est une variante d'échecs à trois joueurs jouée sur un plateau hexagonal de 96 cases, chaque joueur disposant des 16 pièces classiques (soit 48 pièces au total). Le projet consiste à concevoir et implémenter ce jeu en appliquant une modélisation orientée objet et trois design patterns, dont le *Modèle-Vue-Contrôleur* imposé par le sujet.

Les difficultés principales identifiées en phase d'analyse sont : (i) la représentation des 96 cases du plateau Yalta, qui n'est pas une grille cartésienne classique et impose une coordonnée à trois composantes (*secteur, colonne, rang*) ; (ii) les règles de déplacement des pièces lorsqu'elles franchissent la frontière entre deux secteurs, en particulier pour les fous et les reines dont les diagonales « tournent » ; (iii) la gestion d'une partie à trois joueurs, avec l'élimination d'un joueur maté et la poursuite de la partie entre les deux restants.

2 Choix de conception

2.1 Architecture MVC

L'application est structurée en trois couches étanches :

- **Modèle** : `Plateau`, `Partie`, `Piece` et ses sous-classes, `Coup`. Ne dépend d'aucune bibliothèque graphique, ce qui le rend testable unitairement et réutilisable pour brancher l'IA.
- **Vue** : `FenetrePrincipale`, `VuePlateau` (un `QGraphicsScene`), `VuePiece`. Se contente de rendre l'état et d'émettre des signaux Qt lorsqu'une case est cliquée.
- **Contrôleur** : `ContrôleurJeu` reçoit les signaux de la Vue, valide les coups auprès du Modèle et propage les mises à jour.

Le pattern est renforcé par le mécanisme *signals/slots* de Qt, qui joue le rôle d'un Observer intégré : toute modification du modèle émet un signal que la vue écoute pour se redessiner, sans couplage direct.

2.2 Strategy pour les déplacements

Chaque pièce délègue le calcul de ses coups possibles à une implémentation de l'interface `IStrategieDeplacement` (`DeplacementPion`, `DeplacementTour`, etc.). Cela évite un `switch` géant sur le type de pièce dans le moteur, et — surtout — préparera l'intégration future de stratégies de décision IA (`MinMax`, `AlphaBeta`) qui remplaceront `ContrôleurJeu` comme source de coups pour un joueur donné sans modification du modèle.

2.3 Factory pour la création des pièces

`PieceFactory` centralise deux responsabilités : instancier une pièce à partir de son type et de sa couleur (avec injection de la bonne Strategy), et produire la disposition initiale des 48 pièces sur les trois camps Yalta. Cette centralisation évite la duplication de la logique de câblage et facilite la gestion de la *promotion du pion* qui, concrètement, demande de remplacer une pièce par une autre au même emplacement.

3 Diagrammes

Trois diagrammes UML accompagnent cette analyse (dossier `docs/diagrammes/`) :

- **Diagramme de classes** — structure statique complète avec les trois patterns mis en évidence ;

- **Diagramme d'états-transitions** — cycle d'un tour de jeu, avec les états terminaux spécifiques à la variante 3 joueurs ;
- **Diagramme de cas d'utilisation** — acteurs et interactions attendues, avec l'acteur *Joueur IA* déjà identifié pour l'itération suivante.

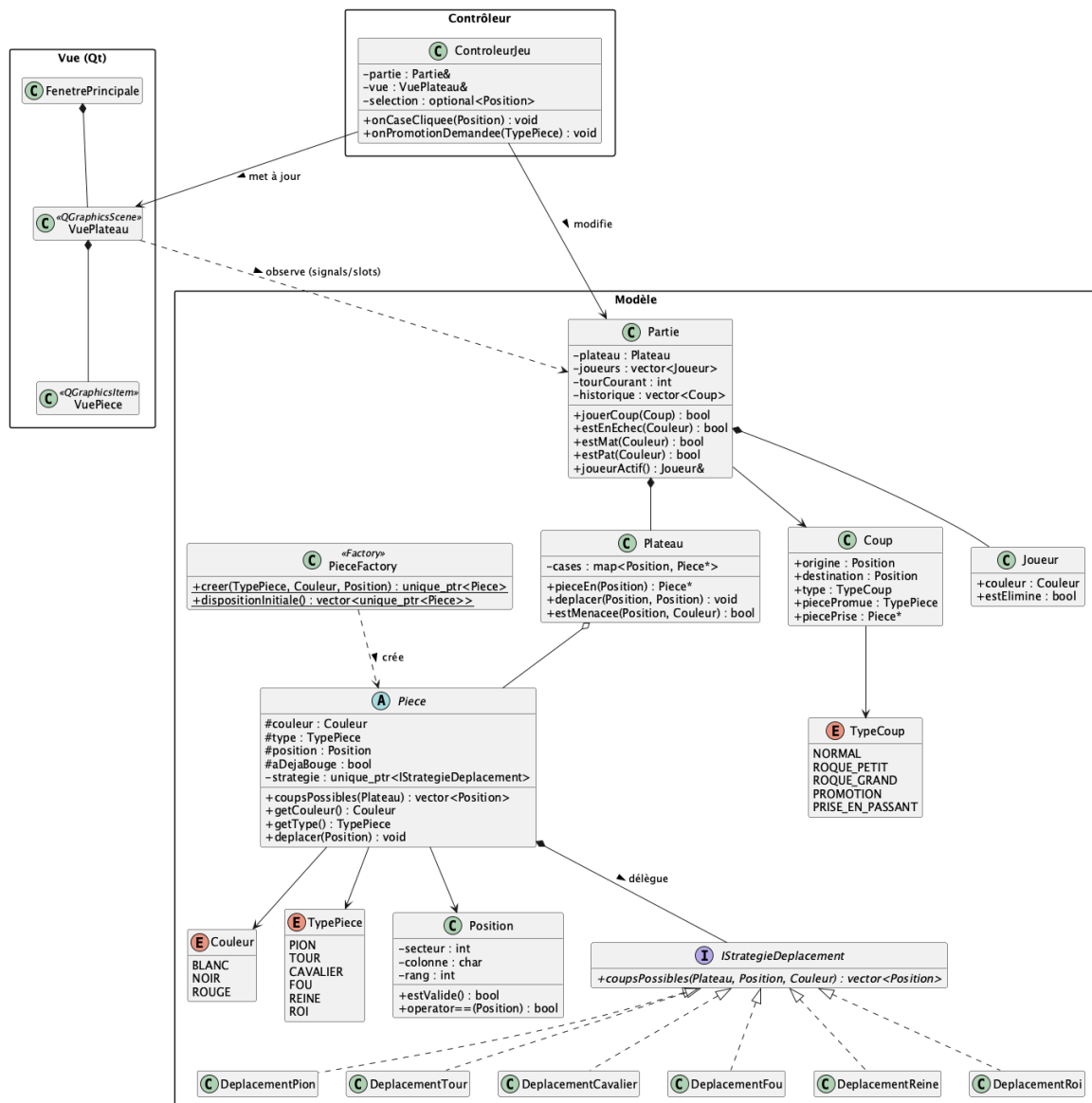


Figure 1: Diagramme de classes simplifié (MVC + Strategy + Factory).

4 Fonctionnalités implémentées

- moteur complet avec les six types de pièces (pion, tour, cavalier, fou, reine, roi) et leurs règles de déplacement via Strategy ;
- plateau à 96 cases et disposition initiale des 48 pièces des 3 camps, construite par la *PieceFactory* ;
- tour de jeu cyclique entre les trois joueurs, détection d'échec, de mat (entraînant l'élimination du joueur) et de pat ;
- coups spéciaux : roque court et long, promotion du pion, prise en passant (API *Partie::tenterRoque*, *Partie::promouvoir*, *Partie::priseEnPassantPossible*) ;
- interface graphique Qt 6 basée sur *QGraphicsScene* avec sélection au clic, affichage des coups légaux en surbrillance, et rafraîchissement par signaux/slots ;
- couverture de test *GoogleTest* sur les déplacements et les cas particuliers du moteur.

5 Difficultés rencontrées

La principale difficulté a été la modélisation de la géométrie du plateau Yalta : la représentation à base de *(secteur, colonne, rang)* adoptée ici est volontairement simplifiée et ne gère pas encore les transitions entre secteurs pour les pièces de longue portée (fous, reines). Cette limite, identifiée dès la conception, sera levée en introduisant une fonction *voisin(Position, direction)* consultée par les Strategy concernées.